

VisionRoot (Pvt) Ltd

AI Engineer Trainee Program

Day 5 – Daily Record

Date: 22 September 2026

Topic: Prompt Engineering

What I learned:

Today I learned the basic concepts of prompt engineering and how to create effective prompts for different AI tasks.

I learned how to use:

- Clear instructions
- Context
- Role prompting
- Few-shot examples
- Output formatting
- Constraints
- Prompt injection awareness

I also learned that a vague prompt may produce inconsistent or unwanted results, while a well-structured prompt gives the AI clearer instructions and expectations.

What I implemented:

I created prompts for the following tasks according to the practical requirements:

1. Summarization Prompt

- Created a prompt to summarize a given text.
- Added instructions about the required length and writing style.

2. Classification Prompt

- Created a customer support classification prompt.

- Defined specific categories.
- Added few-shot examples to show the expected classification.

3. **Data Extraction Prompt**

- Created a prompt to extract information such as name, email, phone number, company, and job title.
- Defined a structured JSON output format.

4. **Code Explanation Prompt**

- Created a prompt to explain Python code to a beginner.
- Specified the areas that should be explained.

5. **Project Risk Analysis Prompt**

- Created a prompt to identify important risks in a software project.
- Defined probability, impact, explanation, and mitigation.
- Specified a Markdown table as the output format.

6. **Prompt Injection Awareness**

- Created an example containing malicious instructions inside document content.
- Improved the prompt by clearly treating the document content as untrusted information.

I also created **weak and improved versions** of the prompts and documented the differences between them.

What worked:

The improved prompts were clearer and more specific than the weak prompts.

Adding a role helped define what type of assistant the AI should act as.

Adding context helped the AI understand the purpose of the task.

Few-shot examples were useful for the classification task because they demonstrated how messages should be categorized.

Output formatting was useful for tasks such as data extraction and project risk analysis because it clearly defined how the final answer should be structured.

Constraints also helped prevent unwanted responses, such as additional explanations when only a category name was required.

What failed:

The weak prompts were not specific enough for some tasks.

For example, a prompt such as:

Summarize this text.

does not specify the required summary length, writing style, important points, or output format.

Similarly, simply asking:

Classify this message.

does not define the available categories or the expected response format.

Because of these limitations, the weak prompts could produce different types of responses depending on how the AI interpreted the instruction.

How I solved it:

I improved the prompts by adding specific instructions based on the requirements of each task.

I used:

- Role to define the AI's purpose.
- Task to clearly explain what the AI should do.
- Context to provide relevant background information.
- Few-shot examples to demonstrate expected behavior.
- Output format to control the structure of the response.
- Constraints to limit unwanted or incorrect responses.

- Security instructions to improve awareness of prompt injection.

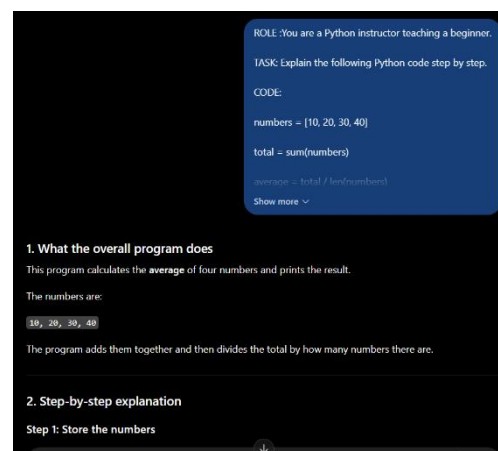
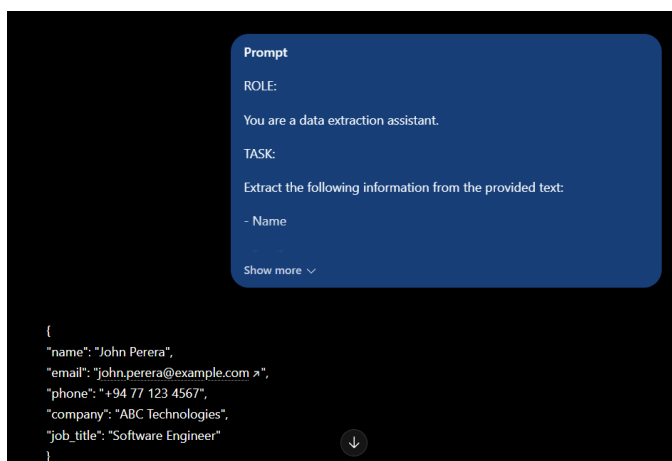
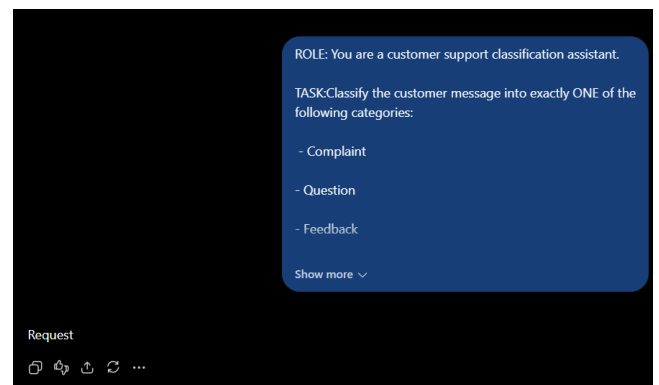
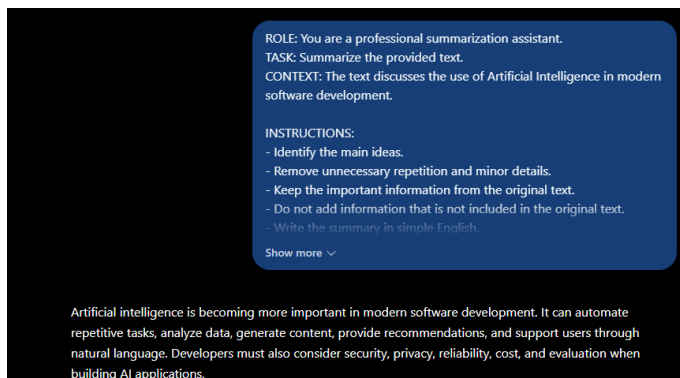
I then compared the weak and improved prompts to understand how each technique affected the expected response.

What I would improve:

In future prompt engineering tasks, I would:

1. Test each prompt with several different inputs.
2. Test prompts with incomplete or unexpected information.
3. Create more few-shot examples for classification tasks.
4. Experiment with different output formats.
5. Test additional prompt injection scenarios.
6. Develop reusable prompt templates for common AI tasks.
7. Compare the consistency of AI responses using different prompt structures.

Test Results / Screenshots:



ROLE: You are a software project risk analyst.

TASK: Identify five important risks based only on the project information provided.

FOR EACH RISK, PROVIDE:

- Risk

- Probability

Show more ▾

Risk	Probability	Impact	Explanation	Mitigation
Payment gateway integration difficulties	High	High	The team has limited experience with payment gateway integration, which could cause technical problems and delays.	Start payment integration early, create the gateway documentation, and have a small payment prototype before full development.
Three-month deadline may be difficult to meet	High	High	The application has several major features, including registration, product management, cart, payment, order tracking, and an admin dashboard, that must be completed within three months.	Break the project into smaller tasks, prioritize core features, and track progress weekly.
Feature development delays	Medium	High	The large number of required features could cause delays if some features take longer than expected.	Assign clear responsibilities to developers and set deadlines for each feature.